НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ЮРИДИЧНА АКАДЕМІЯ»

Факультет кібербезпеки · Кафедра інформаційних технологій

Лабораторна робота 8

Документування архітектури та захист проєкту

Варіант 12 · Управління автопарком та маршрутами

Черниченко Владислав · Коцюба Максим

FleetFlow · 27 жовтня 2026 року

Підсумок системи

FleetFlow - локальна система обліку автопарку і планування рейсів. Поточна реалізація містить браузерну панель, вхід, два незалежні API з SQLite та HTTP bridge. У ЛР8 код системи не змінюється: впорядковано результати попередніх робіт, рішення, вимірювання й обмеження.

У ЛР6 досягнуто p95 470.2 мс при 100 користувачах для першої сторінки 50 активних рейсів; при 200 користувачах - 996.3 мс. У ЛР7 проходять 68 перевірок у двох етапах і негативний експеримент з реальним exit 1.

Репозиторій: https://github.com/VladyslavCh412/labs. Попередні каталоги lab, lab3, lab4, lab5 переглянуто; нові матеріали підготовлено до розміщення в lab6–lab8. Хмарний запуск Actions не заявляється.

Навігація пакетом

1 Огляд і глосарій; 2 C4-контекст; 3 контейнери; 4 компоненти Trips; 5 журнал ADR; 6 атрибути якості; 7 дані та безпека; 8 тести; 9 борг та еволюція; 10 план захисту і джерела. Деталізовані докази знаходяться у verification/; редаговані тексти ADR - docs/adr/; презентація - FleetFlow_Захист.pptx.

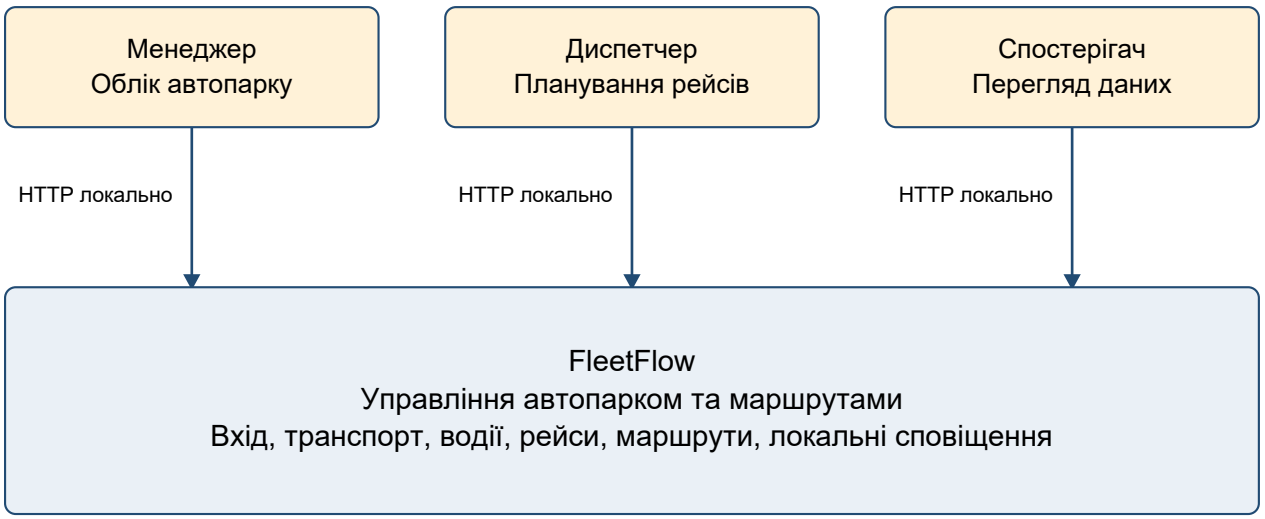
1 Межі системи та глосарій

Менеджер веде облік транспорту й водіїв; диспетчер створює маршрути та планує рейси; спостерігач переглядає дані. Fleet володіє Vehicle і Driver; Trips - Route, Trip, локальними сповіщеннями та похідною проєкцією VehicleSnapshot. Dashboard пересилає HTTP, а не читає БД.

Термін	Однозначне значення у прототипі
Транспортний засіб / Vehicle	Обліковий запис автомобіля; статус available/in_trip/maintenance; власник Fleet.
Водій / Driver	Особа з active та license_valid; допуск перевіряється перед створенням рейсу.
Маршрут / Route	Упорядковані щонайменше дві непорожні точки; це не розрахунок дороги картографічним API.
Рейс / Trip	План перевезення з ідентифікаторами ТЗ, водія й маршруту.
Проєкція	Read-only копія id/plate/status/version у Trips, оновлювана подіями Fleet.
Подія	Envelope з event_id; VehicleStatusChanged або TripCreated.
Outbox / Inbox	Локальні журнали для надійного надсилання / ідемпотентного приймання.
ТО	Стан технічного обслуговування; початок заборонено під час активного рейсу.
Телематика / GPS-трек	Терміни початкового задуму ЛР1; у цьому прототипі немає GPS ingestion.

2 Контекст C4

C4 L1 Контекст реалізованої системи



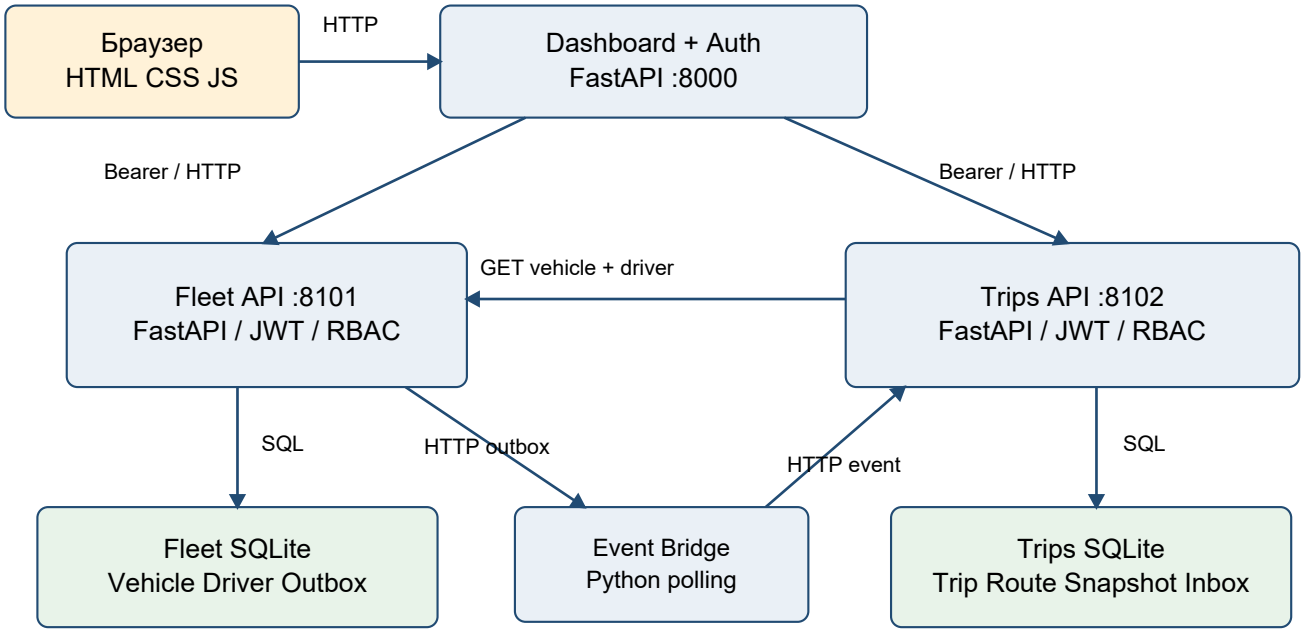
Легенда: жовтий - людина; синій - програмна система.

GPS, зовнішні карти та SMS не реалізовані і не показані як чинні інтеграції.

Контекст показує FleetFlow як єдину систему та три типи користувачів. Початкові карти, GPS і SMS з ЛР1 залишаються планом, а не чинними інтеграціями. Реалізоване сповіщення - локальний запис у notification_inbox.

3 Контейнери C4

C4 L2 Контейнери та власники даних



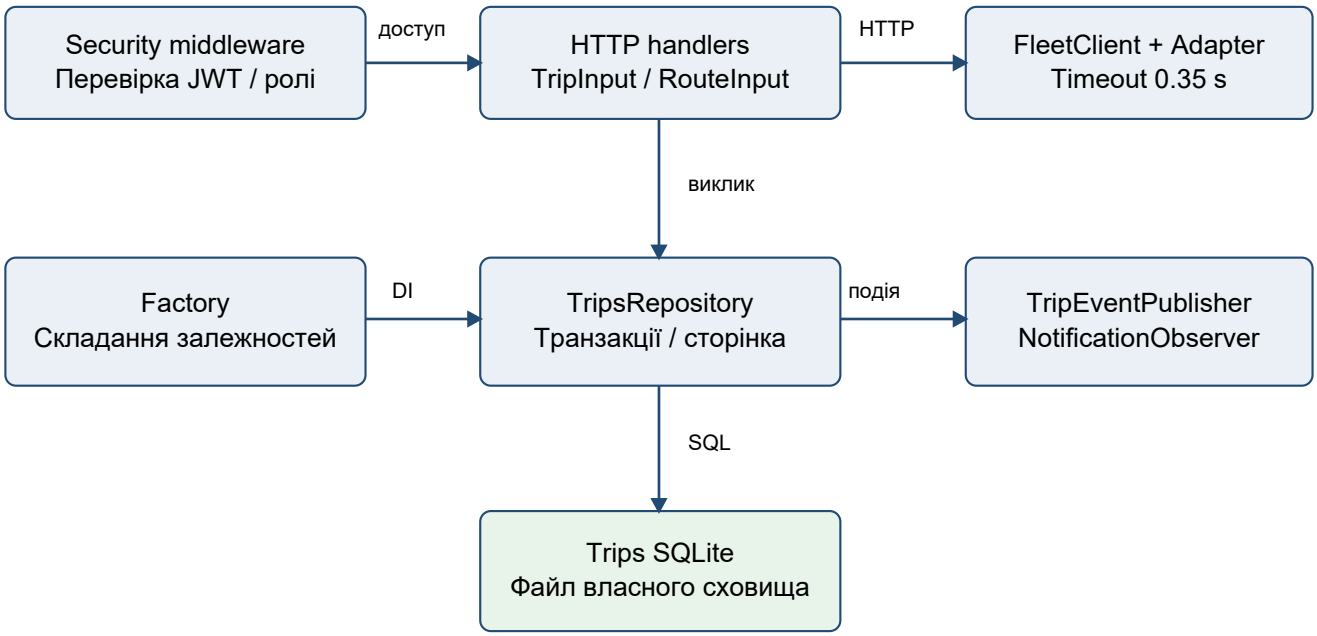
Легенда: жовтий - клієнт; синій - процес; зелений - сховище; стрілки - протокол.

Dashboard на 8000 містить login endpoint і статичний UI. Fleet на 8101 та Trips на 8102 - окремі Uvicorn-процеси. Trips синхронно читає Fleet із timeout 0.35 с та службовою ідентичністю. Bridge опитує outbox раз на 0.5 с, доставляє подію й ACK тільки після успіху.

На діаграмі HTTP - поточний loopback-стенд. Для зовнішнього deployment потрібний HTTPS. Міжсервісні події доставляються через HTTP, окремий брокер зараз не використовується. SQL-зв'язки існують лише між сервісом і його власним сховищем.

4 Компоненти Trips C4

C4 L3 Компоненти контейнера Trips API



Легенда: синій - компонент Trips; зелений - власний файл БД; стрілки - залежності.

Security middleware перевіряє principal до handlers. Pydantic перевіряє форму HTTP body. Factory складає repository, FleetClient та adapter. Репозиторій зберігає Trip і TripCreated в одній локальній транзакції. Observer відокремлює створення локального сповіщення. Domain не залежить від HTTP або SQLite.

Найскладніший контейнер обрано через поєднання синхронного запиту, власної транзакції, outbox, споживання Fleet-події і paginated read. Деталізація всіх контейнерів класами не потрібна для пояснення меж.

5 Журнал архітектурних рішень

Оригінальні ADR-001/002 з актуального lab1.docx і ADR-003 з lab3.docx збережено в docs/adr/*-original.txt без переписування. Статус ADR-001 для поточного deployment відображено в індексі як замінений ADR-003. ADR-004/005 ретроспективно фіксують реалізовані рішення ЛР4/5; дата їх запису - 27.10.2026, не дата вигаданого минулого коміту.

ADR 001 Модульний моноліт

Статус: Замінено для виконуваної системи ADR-003; оригінал ЛР1 збережено. Контекст: Невеликий прототип; проста локальна експлуатація.

Альтернативи: Моноліт; моноліт з worker; 2–3 сервіси.

Рішення: Початкове багатошарове модульне ядро.

Наслідки: Локальні транзакції прості; пізніше межі Fleet/Trips виділено для ЛР3.

ADR 002 FastAPI та незалежний SQLite

Статус: Прийнято; актуальна версія lab1.docx. Контекст: Потрібен локальний REST API без адміністрування серверної БД.

Альтернативи: FastAPI/SQLite; DRF/PostgreSQL; Spring/PostgreSQL.

Рішення: Python, FastAPI, Pydantic, SQLite, Uvicorn, pytest.

Наслідки: Простий запуск; конкурентний запис і масштабування обмежені.

ADR 003 Fleet і Trips як окремі контексти

Статус: Прийнято; замінює ADR-001 щодо deployment. Контекст: ЛР3 потребує власників даних і досліду відмови.

Альтернативи: Моноліт; два контексти; 5+ мікросервісів.

Рішення: Fleet володіє ТЗ і водіями; Trips володіє маршрутами та рейсами. HTTP GET + outbox.

Наслідки: Незалежне володіння; немає спільної ACID-транзакції; потрібні timeout та ідемпотентність.

ADR 004 Патерни для наявних залежностей

Статус: Прийнято; ретроспективний запис рішень ЛР4. Контекст: Шлюз містив умовну маршрутизацію; адаптація JSON та створення залежностей були змішані; сповіщення жорстко зв’язані.

Альтернативи: Залишити if/elif і прямі виклики; узагальнити весь застосунок.

Рішення: Strategy, Adapter, Factory, Observer у конкретних проблемних місцях.

Наслідки: Підміна залежностей і адитивне розширення; додаткові класи виправдані регресійними тестами.

ADR 005 Дані та подієва проєкція

Статус: Прийнято; ретроспективний запис ЛР5. Контекст: Потрібні зворотно сумісні зміни номера та читання стану Fleet у Trips.

Альтернативи: Спільна БД; синхронне копіювання; outbox/inbox.

Рішення: Чотири SQL-міграції; cache TTL 30 с; event_id + version; bridge з ACK після успіху.

Наслідки: Кінцева узгодженість; кеш локальний; під час outage верхня межа lag не гарантована.

ADR 006 Токени та права у власників

Статус: Прийнято в ЛР6. Контекст: ЛР5 дозволяла зміни без автентифікації; gateway можна обійти.

Альтернативи: Лише gateway; JWT у кожному сервісі; зовнішній IdP.

Рішення: Локальні Argon2id-облікові дані, HS256 JWT 15 хв і RBAC у сервісах.

Наслідки: Повторна перевірка; взаємна довіра ключу; TLS/IdP потрібні перед deployment.

ADR 007 Обмежена проєкція сторінки рейсів

Статус: Прийнято в ЛР6. Контекст: fetchall матеріалізує весь набір для першої сторінки, що підтверджено Server-Timing.

Альтернативи: Кеш усіх сторінок; LIMIT/OFFSET; нова БД.

Рішення: Обмежити вибірку SQL LIMIT та не дублювати JSON-перетворення готової проєкції.

Наслідки: p95 поліпшився; великий OFFSET і конкурентні записи потребують окремого аналізу.

ADR 008 Архітектурні перевірки у CI

Статус: Прийнято в ЛР7. Контекст: Ручний контроль імпортів і контрактів не запобігає регресії.

Альтернативи: Ручний review; локальні тести; обов’язковий конвеєр.

Рішення: AST, контракт consumer/provider, domain coverage ≥95%, окремий HTTP p95 gate.

Наслідки: Збірка може впасти через повільний runner; поріг не послаблювати, умови дослідити.

ADR 009 Документація фактичної системи

Статус: Прийнято в ЛР8. Контекст: Початковий контекст мав GPS, карти та зовнішні сповіщення, яких немає у прототипі.

Альтернативи: Показувати задум як реалізацію; явно відділити факт і план.

Рішення: Три C4-рівні для поточного коду; таблиця всіх восьми NFR із невиміряними пунктами.

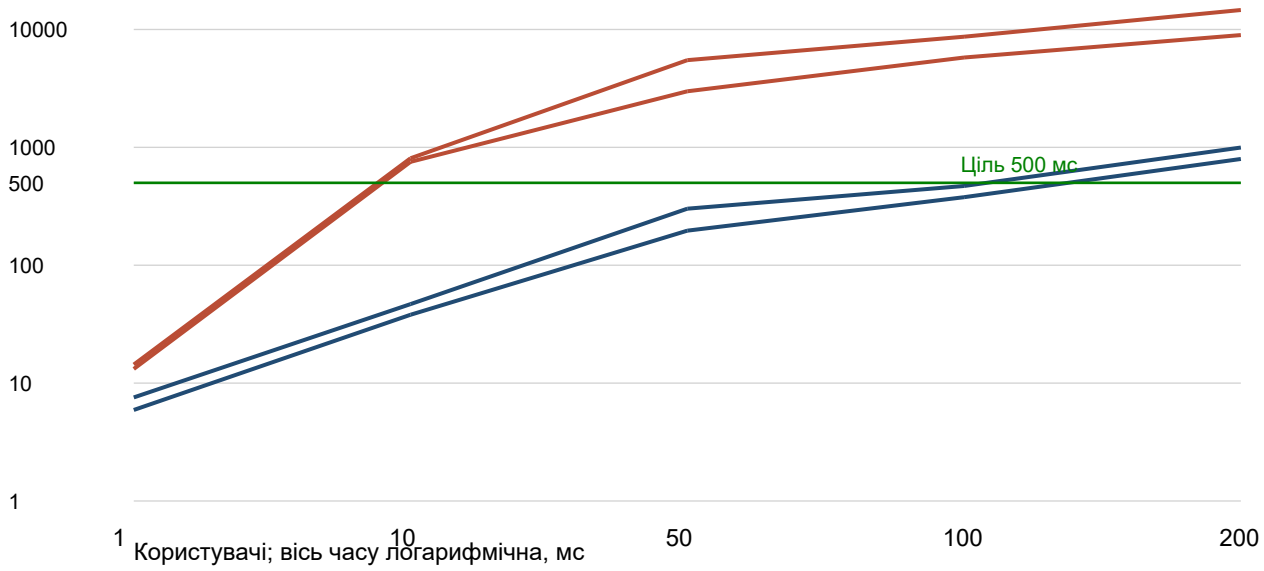
Наслідки: Чесна межа висновків; документацію оновлювати разом із кодом.

6 Атрибути якості та фактичні результати

Атрибут і сценарій ЛР1	Ціль	Факт	Висновок
Продуктивність / список рейсів	p95 ≤500 мс, 100 users, 50 рядків	470.2 мс у ЛР6; 405.5 мс у gate ЛР7	Виконано на локальному стенді; при 200 users порушено
Продуктивність / побудова маршруту	95% ≤3 с, до 20 точок	Є збережений Route; зовнішній routing API відсутній	Не реалізовано / не виміряно
Надійність / GPS	50 msg/s; ≥99.5%; lag ≤10 с	Ідемпотентні бізнес-події перевірені; GPS ingest відсутній	Не підтверджено; це інший сценарій
Надійність / відновлення	RTO ≤30 хв; RPO ≤5 хв	SQLite-перезапуск працює; backup/DR-експеримент не проводився	Не підтверджено
Безпека / відмова доступу	100% відмов; аудит ≤2 с	Негативні кейси на обох API; audit до відповіді	Виконано для перевірених класів токенів і ролей
Безпека / зміна ролей	100% змін ролі аудіюються	UI/API адміністрування ролей немає	Не реалізовано
Підтримуваність / правило ТО	Зміна ≤4 год; зелені тести	Domain незалежний; 100% line coverage; час зміни людиною не виміряно	Частково підтверджено
Інтеграція / збій сповіщень	API ≥99% за 10 хв; retry ≤1 хв	Outbox/inbox і replay перевірені; 10-хв тесту зовнішнього провайдера немає	Частково; бізнес-події не доводять зовнішній SLA

Вісім сценаріїв узято з актуального lab1.docx. Невиміряні показники не підмінено мікробенчмарком кешу або доменним тестом. Висновок про 100 користувачів стосується короткого прямого HTTP читання Trips, а не наскрізного браузера через Dashboard і не конкурентного запису.

HTTP час відповіді до і після оптимізації



Червоний - до; синій - після; суцільна - p95, пунктир - p50.

7 Архітектура даних і захист

Сутність	Власник	Спосіб використання іншими
Vehicle / Driver / vehicle_outbox	Fleet	Trips отримує API; snapshot копіює лише потрібні поля подією.
Route / Trip / TripCreated outbox	Trips	Dashboard використовує HTTP API; посилання на Fleet - лише ID.
VehicleSnapshot / fleet_event_inbox	Trips	Похідне read-only представлення; event_id захищає від повторів, version - від старих подій.
Локальні користувачі та ключ	Auth / setup_local	API перевіряють Bearer; пароль не передається до Fleet/Trips.

Міграції Fleet: 001_expand додає registration_number; 002_dual_write підтримує паралельне значення; 003_switch_reads переводить читання; 004_contract прибирає стару plate. API зберігає ім'я plate. Кеш Vehicle за ID має TTL 30 с й інвалідацію після PATCH; кеш окремий для кожного процесу.

Подія VehicleStatusChanged створюється в тій самій транзакції, що зміна статусу. Bridge використовує at-least-once; повтор event_id не змінює результат. У нормі lag залежить від poll 0.5 с і HTTP, але під час outage гарантованого максимуму немає. Номер ТЗ у snapshot може оновитися лише наступною подією статусу: окрема подія RegistrationChanged не реалізована.

Модель загроз і матриця ролей наведені повністю у звіті ЛР6. Авторизація в сервісі-власнику захищає прямий порт. Внутрішні події мають окрему роль bridge. Публікація секретів запобігається генерацією тільки в data/ та виключенням data/ із Git/ZIP.

8 Стратегія тестування

68 перевірок: 27 модульних, 30 інтеграційних, 7 контрактних, 4 fitness-кейси. Два доменні модулі мають 100% line coverage; це не покриття всього застосунку. Незалежні БД, міграції та outbox перевіряються на тимчасових даних.

Функціональний етап конвеєра: 67 passed; performance етап: 1 passed. Негативний експеримент: forbidden domain import sqlite3, exit 1. GitHub Actions-конфігурація готова, але новий хмарний запуск не виконувався. Докази: pipeline.log, test_results.xml, fitness_results.xml, coverage.json, test_audit.json, negative_pipeline.json, live_smoke.json.

9 Технічний борг та еволюція

Спрощення	Чому прийняте	Спостережуваний тригер
SQLite, один процес на сервіс	Простий переносний локальний стенд	>100 одночасних користувачів або регулярні write-lock помилки: оцінити PostgreSQL
RAM-кеш у процесі	TTL 30 с достатній для демонстрації	Другий Fleet worker: запровадити спільну інвалідацію / розподілений кеш
Polling bridge і відсутність DLQ	Немає залежності від брокера	Lag >10 с протягом 5 хв або >50 events/s: брокер, retry backoff, DLQ
HS256, локальні облікові записи, HTTP	Тільки localhost і випадкові навчальні паролі	Перед першим зовнішнім або персональним deployment: TLS, IdP, асиметричні ключі, ротація, відзив
Немає резервування ТЗ для паралельних рейсів	Поточний сценарій лише планує; бізнес-правила початку відокремлені	Перед увімкненням HTTP start/complete: атомарне резервування й компенсація
Короткий load test та немає DR	Перевіряє regression gate, а не SLA	Перед обіцянкою SLA: 30-хв soak, write profile, backup/restore та аварійний тест

Крок	Що виконати	Порядок і тригер
1	TLS + зовнішній IdP, ключі перевірки, рольове адміністрування й аудит; backup/restore	Перед відкриттям доступу поза localhost або використанням персональних даних.
2	Серверна СУБД, cursor pagination, спільний кеш; профілі читання/запису	Якщо p95 >500 мс при потрібному робочому навантаженні; почати з виміряної ділянки.
3	Брокер, backoff/DLQ, GPS ingestion та зовнішні адаптери	Lag >10 с протягом 5 хв, >50 events/s або затверджена бізнес-потреба GPS/карт.

10 Захист до десяти хвилин

Час	Матеріал	Основне пояснення
0:00–1:00	Проблема та C4 L1	Менеджер, диспетчер, спостерігач; поточна межа системи.
1:00–2:30	C4 L2/L3	Власники даних, HTTP, outbox та залежності.
2:30–4:00	ADR-003 і ADR-006	Чому два контексти; чому права перевіряються у власників.
4:00–5:30	Демонстрація	Вхід manager; dispatcher прямий PATCH → 403; створення рейсу; bridge.
5:30–7:00	Навантаження	p50/p95, однакові умови, SQL LIMIT та порушення на 200 users.
7:00–8:30	Конвеєр	68 перевірок; 100% domain line coverage; навмисне порушення → exit 1.
8:30–9:30	Борг і план	TLS/IdP, масштабування даних, broker/GPS за тригерами.
9:30–10:00	Висновок	Що підтверджено вимірюваннями; які NFR ще не доведені.

Розподіл підготовки: Владислав пояснює вимірювання, fitness і еволюцію; Максим - C4, ADR і модель доступу. Для двох учасників ролі архітектора, аналітика та інженера якості суміщаються. Перед захистом кожен має самостійно запустити проєкт і пояснити закріплені рішення.

Висновок і джерела

Від первинного модульного моноліту ЛР1 система перейшла до навчальних контекстів Fleet і Trips. Реальний стек FastAPI/SQLite з актуального ADR-002 збережено. Нові роботи додали перевірений доступ, обмежену SQL-проєкцію та автоматичні архітектурні правила. ЛР8 узгоджує документацію з цим кодом і не приписує прототипу GPS, TLS або зовнішній SLA.

Джерела: lab1.docx–lab5.docx, методички ЛР1–8, C:\lab5 і GitHub VladyslavCh412/labs; C4 Model (c4model.com), ADR (adr.github.io), документація FastAPI, OWASP REST Security, pytest та GitHub Actions.